



ELTE EÖTVÖS LORÁND
UNIVERSITY

Algoritmusok és Adatszerkezetek I.

Pusztai Gábor

Algoritmusok és Alkalmazásaik Tanszék
Informatikai Kar
ELTE - Eötvös Loránd Tudományegyetem, Budapest

2026. március 1.

1 Láncolt lista - Kétirányú

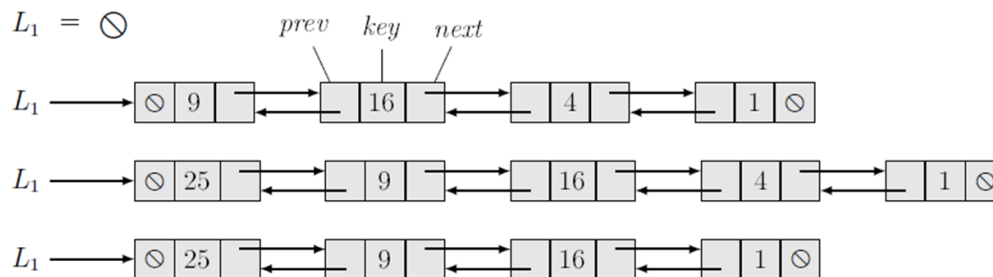
- Egyszerű, kétirányú lista (S2L)
- Fejelemes ciklikus kétirányú lista (C2L)

2 C2L Feladatok

- A rendezett lista párosra és páratlanra szűrve
- Két C2L egyesítése (Uniója)
- Két C2L uniója és metszete
- C2L - Szorgalmi beadandó

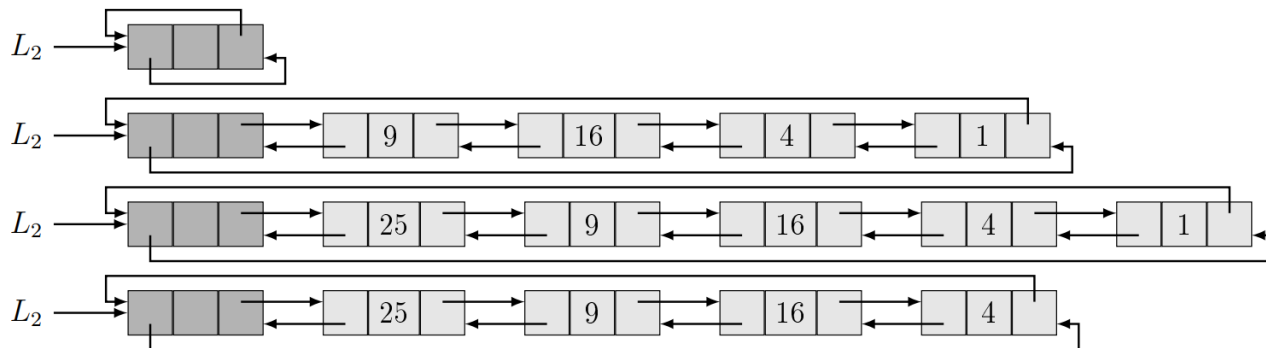
Láncolt lista - Kétirányú

Egyszerű, kétirányú lista (S2L)



- Előny: Könnyen iterálható és "manipulálható".
- S1L-hez hasonlóan az első, belső és utolsó elemek kezelése még mindig különböző kezelést igényel:
 - Egy lista közepén lévő elemnek van előző és következő szomszédja, ami azt jelenti, hogy 4 mutatót kell kezelnünk.
 - Az első elemnek nincs előző szomszédja, míg az utolsó elemnek nincs következője.
 - Üres listába történő beszúrásakor az beszúrandó elemnek nem lesznek szomszédai.

Fejelemes ciklikus kétirányú lista (C2L)



- Minden elem tartalmazza az előző és a következő mutatót.
- A „ciklikus” jelleg az alábbiakat jelenti:
 - A fej elem előző mutatója mindig a lista utolsó elemére mutat
 - A lista utolsó elemének következő mutatója tehát az első elemre mutat

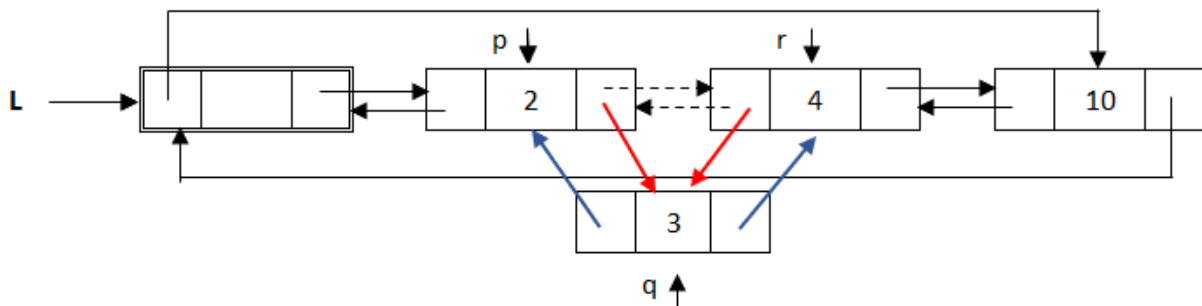
Ciklikus kétirányú lista (C2L) műveletek, E2 Node típus

E2
+ <i>prev, next</i> : E2* // refer to the previous and next neighbour or be this
+ <i>key</i> : $\mathcal{T}$
+ E2() { <i>prev</i> := <i>next</i> := this }

precede(<i>q, r</i> : E2*)	follow(<i>p, q</i> : E2*)
// (* <i>q</i>) will precede (* <i>r</i>)	// (* <i>q</i>) will follow (* <i>p</i>)
<i>p</i> := <i>r</i> → <i>prev</i>	<i>r</i> := <i>p</i> → <i>next</i>
<i>q</i> → <i>prev</i> := <i>p</i> ; <i>q</i> → <i>next</i> := <i>r</i>	<i>q</i> → <i>prev</i> := <i>p</i> ; <i>q</i> → <i>next</i> := <i>r</i>
<i>p</i> → <i>next</i> := <i>r</i> → <i>prev</i> := <i>q</i>	<i>p</i> → <i>next</i> := <i>r</i> → <i>prev</i> := <i>q</i>

unlink(<i>q</i> : E2*)
// remove (* <i>q</i>)
<i>p</i> := <i>q</i> → <i>prev</i> ; <i>r</i> := <i>q</i> → <i>next</i>
<i>p</i> → <i>next</i> := <i>r</i> ; <i>r</i> → <i>prev</i> := <i>p</i>
<i>q</i> → <i>prev</i> := <i>q</i> → <i>next</i> := <i>q</i>

Ciklikus kétirányú lista (C2L) műveletek - precedence



$\text{precede}(q, r : E2^*)$

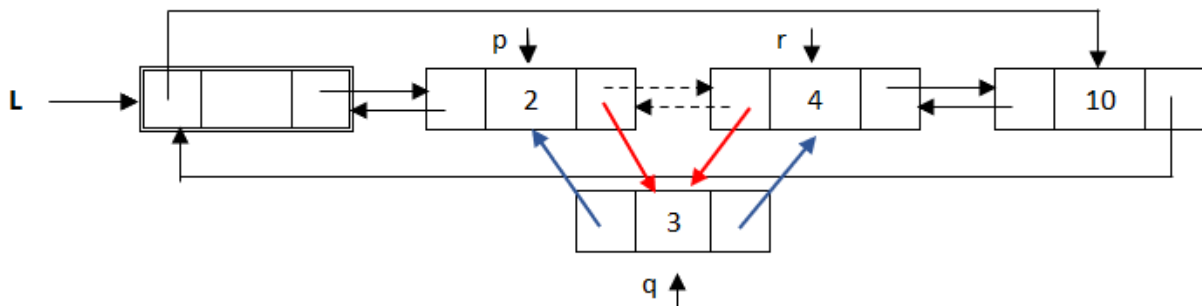
// $(*q)$ will precede $(*r)$

$p := r \rightarrow \text{prev}$

$q \rightarrow \text{prev} := p ; q \rightarrow \text{next} := r$

$p \rightarrow \text{next} := r \rightarrow \text{prev} := q$

Ciklikus kétirányú lista (C2L) műveletek - follow



$\text{follow}(p, q : E2^*)$

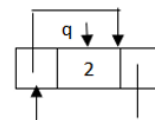
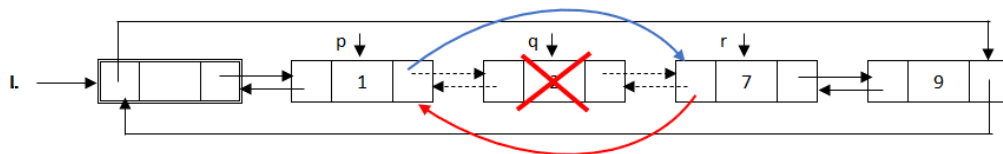
// $(*q)$ will follow $(*p)$

$r := p \rightarrow next$

$q \rightarrow prev := p ; q \rightarrow next := r$

$p \rightarrow next := r \rightarrow prev := q$

Ciklikus kétirányú lista (C2L) műveletek - unlink



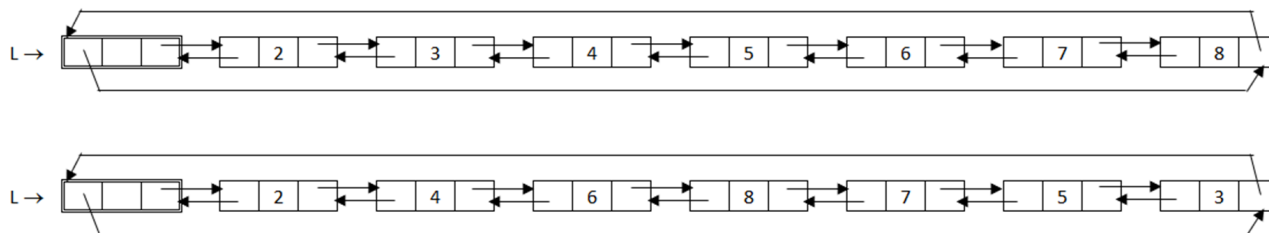
$\text{unlink}(q : E2^*)$

$// \text{ remove } (*q)$
$p := q \rightarrow \text{prev} ; r := q \rightarrow \text{next}$
$p \rightarrow \text{next} := r ; r \rightarrow \text{prev} := p$
$q \rightarrow \text{prev} := q \rightarrow \text{next} := q$

C2L Feladatok

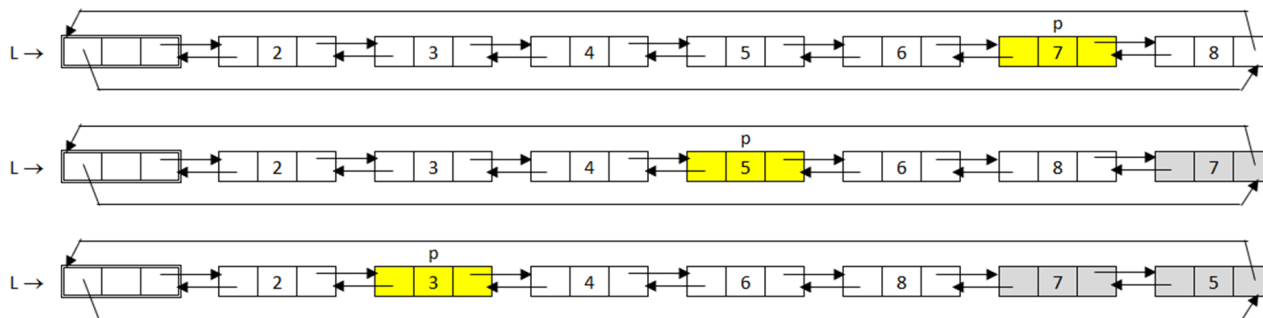
A rendezett lista párosra és páratlanra szűrve

A rendezett lista párosra és páratlanra szűrve



- Tegyük fel, hogy L egy rendezett C2L lista, amely természetes számokat tartalmaz.
- Rendezni kell a listát úgy, hogy a lista „elején” csak páros számok legyenek, a „végén” pedig csak páratlan számok.
- Megjegyzés: A lista mindkét „része” rendezett maradjon.

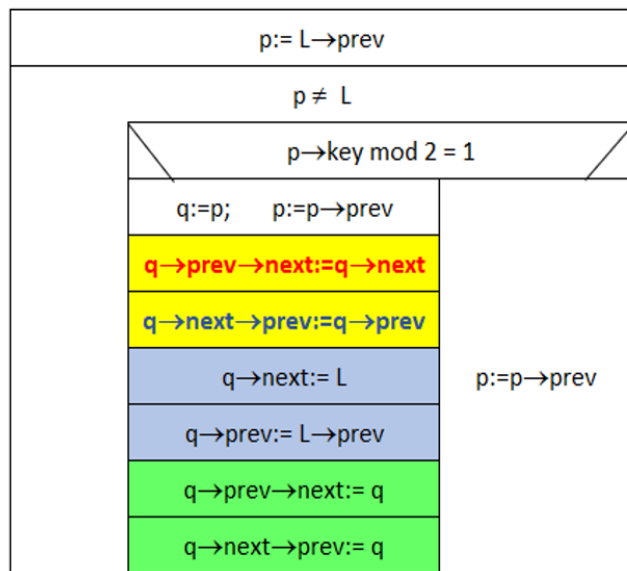
A rendezett lista párosra és páratlanra szűrve



- Iteráljunk visszafelé a listán p-vel.
- Ha a p elem kulcsa páratlan, az elemet kifűzzük, és átláncoljuk a lista végére (a fejelem elé).
- A kiláncoláshoz egy q segéd pointert fogunk használni, p-vel pedig a kiláncolás előtt tovább lépünk.

A rendezett lista párosra és páratlanra szűrve

ReorderC2L(L:E2*)



We only have to do anything if p is odd

Yellow: Linking out the odd element

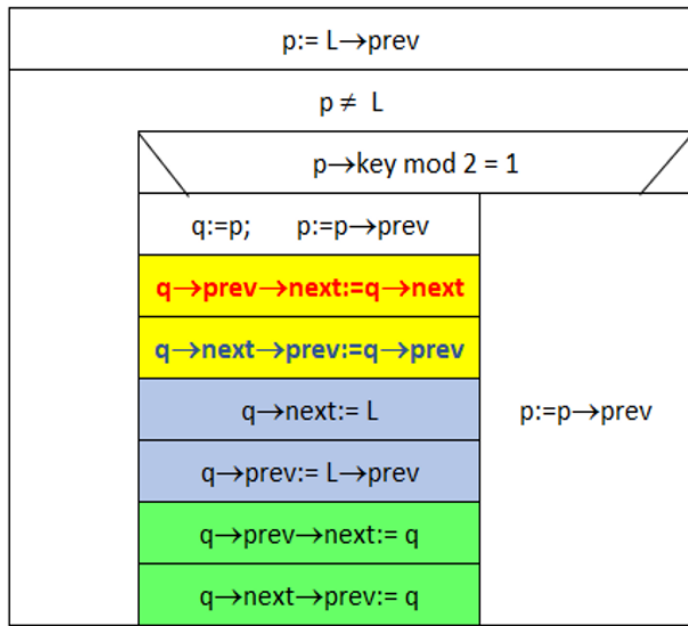
Blue: Inserting q before the header.

Green: Setting the neighbours' pointers

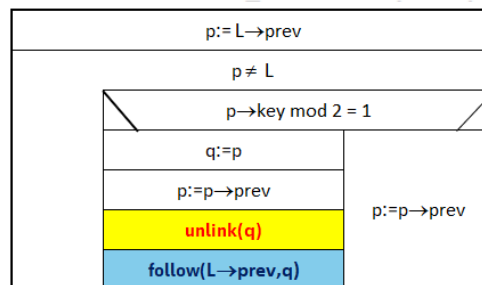
33

A rendezett lista párosra és páratlanra szűrve - egyszerűsített

ReorderC2L(L:E2*)



Simplified ReorderC2L(L:E2*)

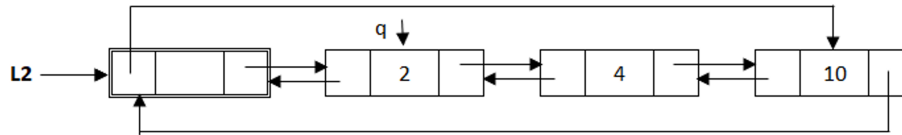
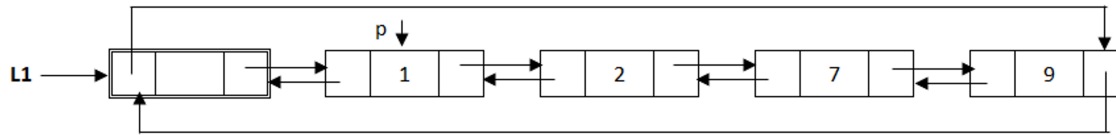


Két C2L egyesítése (Uniója)

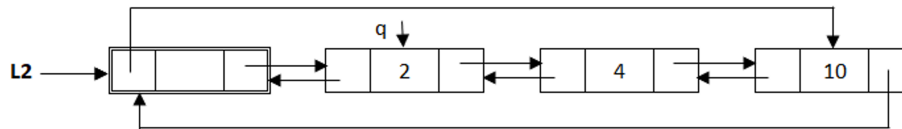
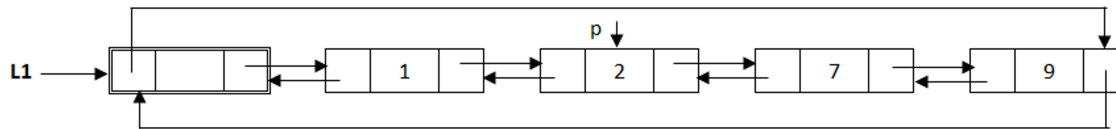
Két C2L egyesítése (Uniója)

- Két lista egyesítése gyakran előforduló feladat, és nem csak állásinterjúkon. A listák rendezett jellege alapvető szerepet játszik ezen algoritmusok hatékonyságában. Mivel már nem tömbökkel dolgozunk, egy elem beszúrása nem igényli a többi eltolását.
- Általában olyan listákkal dolgozunk, amelyek tartalmazznak fejléceket (H1L, C2L).
- Példa: Legyen L1 és L2, amelyek különböző rendezett C2L-ekre mutatnak. Tegyük fel, hogy a kulcsok is egyediek (ezáltal a lista halmaznak tekinthető). A kimenet legyen L1, és a hozzá tartozó lista a két halmaz uniója.
- Ötlet: Minden listának van egy mutatója, amely az elemein iterál. Ezután 3 esetünk van.

Két C2L egyesítése - Eset 1

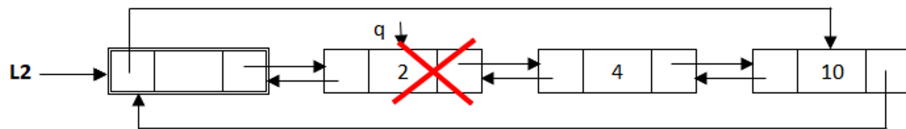
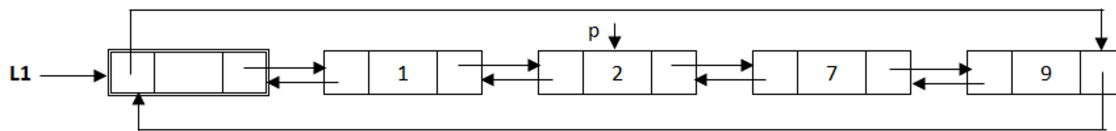


P keeps going, but q remains in place

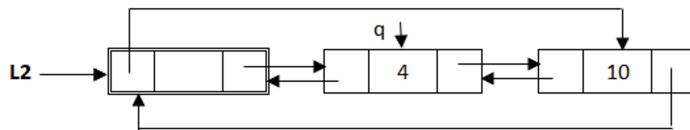
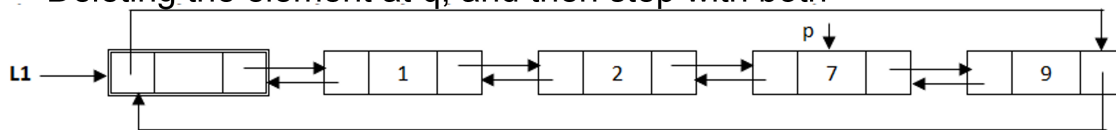


- (1) Az aktuális elem L1-ben kisebb, mint az elem L2-ben, lépünk az L1 mutatójával, de L2-ben helyben maradunk.

Két C2L egyesítése - Eset 2

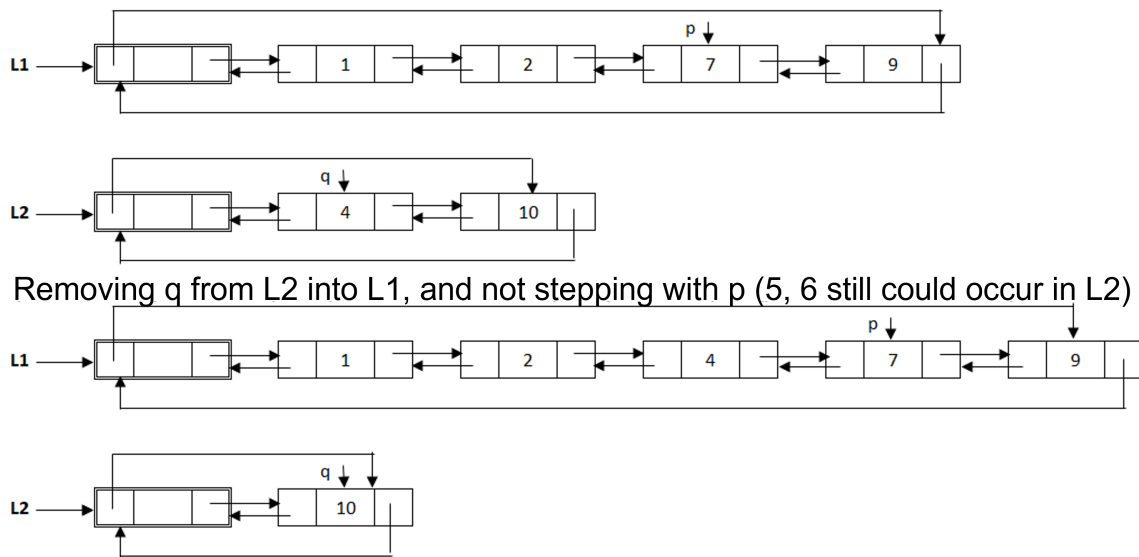


Deleting the element at q , and then step with both



- (2) Az aktuális elem L1-ben megegyezik az aktuális elemmel L2-ben. Eltávolítjuk az elemet L2-ből, és mindkét mutatót léptetjük.

Két C2L egyesítése - Eset 3



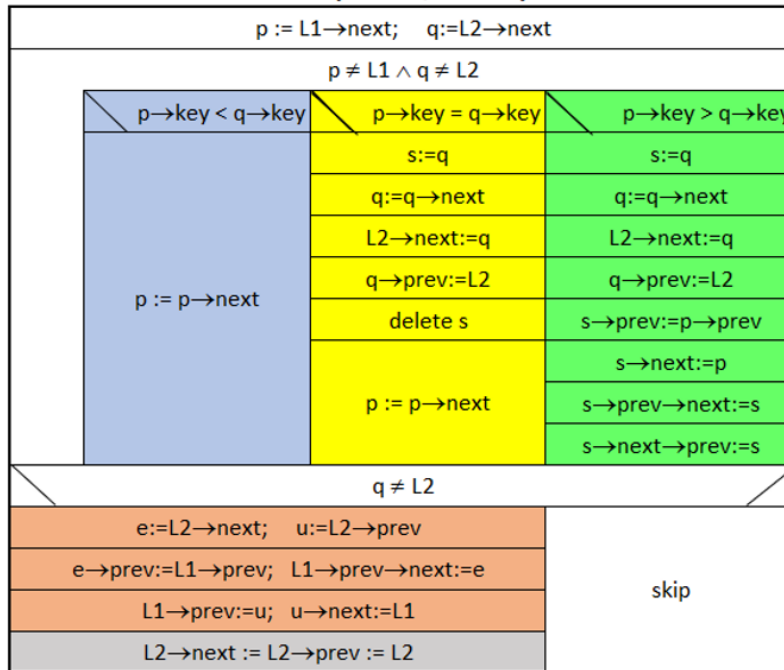
- (3) L1-ben p-nél nagyobb elem van, mint a q-nál lévő elem L2-ben. Ekkor beszúrjuk az elemet q-ból L1-be azáltal, hogy eltávolítjuk L2-ből, és léptetjük q-t, miközben p-t nem mozdítjuk.

Két C2L egyesítése - A lista végén

- Ha $p=L1$, akkor elértük L1 végét, különben ha $q=L2$, akkor L2 végét értük el. (Ez egyidejűleg is előfordulhat, ha mindkét mutatót léptetjük.)
- L2 végére érve azt jelenti, hogy kész vagyunk, nincs több elem, amit esetleg be kellene illeszteni L1-be. Ha viszont L1 végét érjük el, akkor ez azt jelenti, hogy még vannak elemek L2-ben, amelyek nincsenek jelen L1-ben. Ezért a maradék L2-t másolnunk kell L1-be. Ezt megtehetjük egyesével, de konstans időben is!
- Megjegyzés: Az algoritmus bármely pontján az alábbi igaz: $q=L2 \rightarrow \text{next}$. Ez azt jelenti, hogy nincs szükség a q változóra a megvalósításban, de az olvashatóság kedvéért most még használni fogjuk.

Két C2L egyesítése

Unio(L1: E2*, L2: E2*)



- Kék:
 - Az aktuális elem L1-ben kisebb, mint az aktuális elem L2-ben: Csak léptessük p-t.
- Sárga:
 - Az elemek egyenlők. Ez azt jelenti, hogy L1 már tartalmazza azt az elemet, így csak el kell távolítanunk a q-t L2-ből, és a következő elemre kell állítanunk.
- Zöld:
 - Találtunk egy elemet, amely nincs jelen L1-ben, tehát be kell illeszteniünk L2-ből L1-be. Ez ugyanaz, mint az előző, csak most nemcsak eltávolítjuk q-t L2-ből, hanem be is illesztjük L1-be, és nem léptetjük p-t.
- Barna:
 - Az L2 maradék elemeit hozzáadjuk L1-hez.
- Megjegyzés:
 - Egyes utasítások sorrendje lényeges a megfelelő működéshez, de nem mindegyiké. A prev értékének beállítása a next előtt nem okoz problémát, de nem állíthatjuk be a szomszédos elemek értékeit, mielőtt a sajátunkét beállítanánk.

Két C2L egyesítése - Egyszerűsített változat

Unio(L1: E2*, L2: E2*)

$p := L1 \rightarrow \text{next}; \quad q := L2 \rightarrow \text{next}$

$p \neq L1 \wedge q \neq L2$

$p \rightarrow \text{key} < q \rightarrow \text{key}$	$p \rightarrow \text{key} = q \rightarrow \text{key}$	$p \rightarrow \text{key} > q \rightarrow \text{key}$
$p := p \rightarrow \text{next}$	$s := q$	$s := q$
	$q := q \rightarrow \text{next}$	$q := q \rightarrow \text{next}$
	$L2 \rightarrow \text{next} := q$	$L2 \rightarrow \text{next} := q$
	$q \rightarrow \text{prev} := L2$	$q \rightarrow \text{prev} := L2$
	delete s	$s \rightarrow \text{prev} := p \rightarrow \text{prev}$
	$p := p \rightarrow \text{next}$	$s \rightarrow \text{next} := p$
		$s \rightarrow \text{prev} \rightarrow \text{next} := s$
		$s \rightarrow \text{next} \rightarrow \text{prev} := s$

$q \neq L2$

$e := L2 \rightarrow \text{next}; \quad u := L2 \rightarrow \text{prev}$

$e \rightarrow \text{prev} := L1 \rightarrow \text{prev}; \quad L1 \rightarrow \text{prev} \rightarrow \text{next} := e$

$L1 \rightarrow \text{prev} := u; \quad u \rightarrow \text{next} := L1$

$L2 \rightarrow \text{next} := L2 \rightarrow \text{prev} := L2$

skip

Unio(L1: E2*, L2: E2*)

$p := L1 \rightarrow \text{next}; \quad q := L2 \rightarrow \text{next}$

$p \neq L1 \wedge q \neq L2$

$p \rightarrow \text{key} < q \rightarrow \text{key}$	$p \rightarrow \text{key} = q \rightarrow \text{key}$	$p \rightarrow \text{key} > q \rightarrow \text{key}$
$p := p \rightarrow \text{next}$	$s := q$	$s := q$
	$q := q \rightarrow \text{next}$	$q := q \rightarrow \text{next}$
	unlink(s)	unlink(s)
	delete s	precede(s,p)
	$p := p \rightarrow \text{next}$	

$L2 \rightarrow \text{next} \neq L2$

$s := L2 \rightarrow \text{next}$

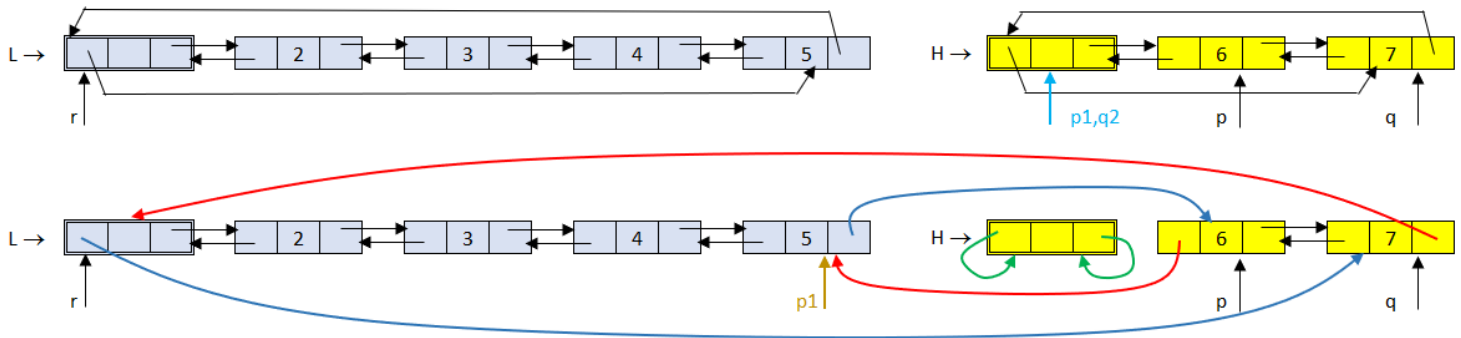
unlink(s)

follow(L1 $\rightarrow$ prev, s)

Két C2L egyesítése - Hatékonyabban

- Hatékonyabbá tehetjük a megoldásunkat, ha amikor főciklusból kilépve, az L2 listában még vannak elemek, azokat nem ciklussal fűzzük át L1 listába, hanem az L2-beli lánc-darabot a két végénél fogva konstans lépésben fűzzük L1 lista végére.
- A splice($p, q, r : E2^*$) egy olyan elemi listaművelet, amely egy C2L egy adott $[p, \dots, q]$ szakaszát eltávolítja, majd az eltávolított szakaszt egy C2L adott $*r$ eleme elé befűzi. (Előfeltétel, hogy $p..q$ szakasz ne tartalmazzon a fejelemet, p után jöjjön q ($p=q$ lehet), valamint $*r$ ne legyen a $p..q$ szakasz egy eleme, de r lehet egy másik C2L listának az eleme.)

Két C2L egyesítése - Hatékonyabban



$\text{append}(L, H : E2^*)$

$H \rightarrow \text{next} \neq H$	
$\text{splice}(H \rightarrow \text{next}, H \rightarrow \text{prev}, L)$	SKIP

$\text{splice}(p, q, r : E2^*)$

$p1 := p \rightarrow \text{prev} ; q2 := q \rightarrow \text{next}$
$p1 \rightarrow \text{next} := q2 ; q2 \rightarrow \text{prev} := p1$
$p1 := r \rightarrow \text{prev}$
$p \rightarrow \text{prev} := p1 ; q \rightarrow \text{next} := r$
$p1 \rightarrow \text{next} := p ; r \rightarrow \text{prev} := q$

Két C2L egyesítése - Hatékonyabban

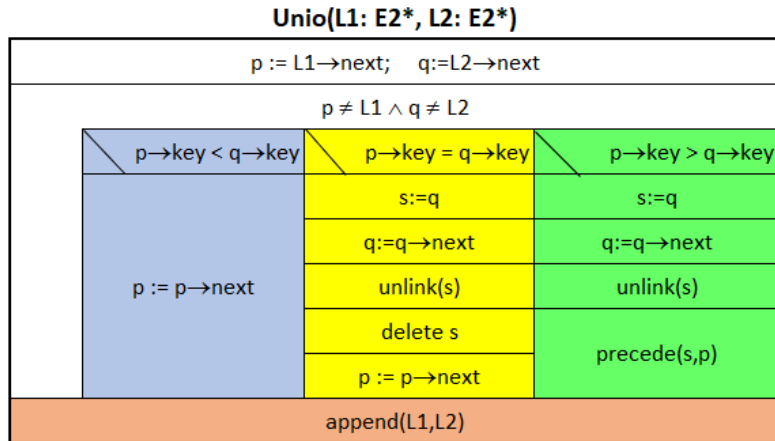
$\text{splice}(p, q, r : E2^*)$

$p1 := p \rightarrow prev ; q2 := q \rightarrow next$
$p1 \rightarrow next := q2 ; q2 \rightarrow prev := p1$
$p1 := r \rightarrow prev$
$p \rightarrow prev := p1 ; q \rightarrow next := r$
$p1 \rightarrow next := p ; r \rightarrow prev := q$

- p1- p bal szomszédja, q2 q jobb szomszédja
- összeláncoljuk p1 és q2 elemeket (szakasz kifűzése)
- p1 az r bal szomszédja, p1 és r közé fűzzük p..q szakaszt
- p és q című elemek összekapcsolása p1 és r elemekkel
- p1 és r című elemek összekapcsolása p és q elemekkel

- Ennek segítségével definálhatjuk az append műveletet, amely egy L C2L lista végére fűzi egy H nem üres C2L lista összes elemét.

Két C2L egyesítése



- Vizsgáljuk meg az összefésülő algoritmusunk műveletigényét!:
- Legyen L1 lista hossza: n , L2 lista hossza m .
- Az algoritmus az egyik listán mindenképp végig iterál, tehát:
- $mT(n, m) = \theta(n)$ abban az esetben, ha L1 minden eleme kisebb, mint L2 első eleme.
- $mT(n, m) = \theta(m)$ abban az esetben, ha L2 minden eleme kisebb, mint L1 első eleme.
- $MT(n, m) = \theta(n + m)$. Például pontosan $n + m$ iterációt hajt végre abban az esetben, ha nincsenek azonos elemek, és L1 utolsó előtti eleme kisebb, a legutolsó pedig nagyobb, mint L2 legutolsó eleme.

Két C2L uniója és metszete

Két C2L uniója és metszete

- Legyenek Hu és Hi szigorúan monoton növekvően rendezett C2L-ek!
- Írjuk meg a $\text{unionIntersection}(Hu, Hi : E2^*)$ eljárást, ami a Hu listába Hi megfelelő elemeit átfűzve, a Hu listában az eredeti listák, mint halmazok unióját állítja elő, míg a Hi listában a metszetük marad!
- Ne allokáljunk és ne is deallokáljunk listaelemeket, csak az listaelemek átfűzésével oldjuk meg a feladatot!
 $MT(n_u, n_i) \in \theta(n_u + n_i)$, ahol a Hu C2L hossza n_u , a Hi C2L hossza pedig n_i . Mindkét lista maradjon szigorúan monoton növekvően rendezett C2L!

Két C2L uniója és metszete

$$H_u \rightarrow [/\text{---}[2]_{\text{---}}[4]_{\text{---}}[6]_{\text{---}}[/\leftarrow H_u$$

$$H_i \rightarrow [/\text{---}[1]_{\text{---}}[4]_{\text{---}}[5]_{\text{---}}[8]_{\text{---}}[9]_{\text{---}}[/\leftarrow H_i$$

$$H_u \rightarrow [/\text{---}[1]_{\text{---}}[2]_{\text{---}}[4]_{\text{---}}[6]_{\text{---}}[/\leftarrow H_u$$

$$H_i \rightarrow [/\text{---}[4]_{\text{---}}[5]_{\text{---}}[8]_{\text{---}}[9]_{\text{---}}[/\leftarrow H_i$$

$$H_u \rightarrow [/\text{---}[1]_{\text{---}}[2]_{\text{---}}[4]_{\text{---}}[6]_{\text{---}}[/\leftarrow H_u$$

$$H_i \rightarrow [/\text{---}[4]_{\text{---}}[5]_{\text{---}}[8]_{\text{---}}[9]_{\text{---}}[/\leftarrow H_i$$

$$H_u \rightarrow [/\text{---}[1]_{\text{---}}[2]_{\text{---}}[4]_{\text{---}}[6]_{\text{---}}[/\leftarrow H_u$$

$$H_i \rightarrow [/\text{---}[4]_{\text{---}}[5]_{\text{---}}[8]_{\text{---}}[9]_{\text{---}}[/\leftarrow H_i$$

$$H_u \rightarrow [/\text{---}[1]_{\text{---}}[2]_{\text{---}}[4]_{\text{---}}[5]_{\text{---}}[6]_{\text{---}}[/\leftarrow H_u$$

$$H_i \rightarrow [/\text{---}[4]_{\text{---}}[8]_{\text{---}}[9]_{\text{---}}[/\leftarrow H_i$$

$$H_u \rightarrow [/\text{---}[1]_{\text{---}}[2]_{\text{---}}[4]_{\text{---}}[5]_{\text{---}}[6]_{\text{---}}[/\leftarrow H_u$$

$$H_i \rightarrow [/\text{---}[4]_{\text{---}}[8]_{\text{---}}[9]_{\text{---}}[/\leftarrow H_i$$

$$H_u \rightarrow [/\text{---}[1]_{\text{---}}[2]_{\text{---}}[4]_{\text{---}}[5]_{\text{---}}[6]_{\text{---}}[8]_{\text{---}}[/\leftarrow H_u$$

$$H_i \rightarrow [/\text{---}[4]_{\text{---}}[9]_{\text{---}}[/\leftarrow H_i$$

$$H_u \rightarrow [/\text{---}[1]_{\text{---}}[2]_{\text{---}}[4]_{\text{---}}[5]_{\text{---}}[6]_{\text{---}}[8]_{\text{---}}[9]_{\text{---}}[/\leftarrow H_u$$

$$H_i \rightarrow [/\text{---}[4]_{\text{---}}[/\leftarrow H_i$$

unionIntersection($H_u, H_i : E2*$)

$q := H_u \rightarrow next ; r := H_i \rightarrow next$		
$q \neq H_u \wedge r \neq H_i$		
$q \rightarrow key < r \rightarrow key$	$q \rightarrow key > r \rightarrow key$	$q \rightarrow key = r \rightarrow key$
$q := q \rightarrow next$	$p := r$	$q := q \rightarrow next$
	$r := r \rightarrow next$	
	unlink(p)	$r := r \rightarrow next$
	precede(p, q)	
$r \neq H_i$		
$p := r ; r := r \rightarrow next ; \text{unlink}(p)$		
precede(p, H_u)		

C2L - Szorgalmi beadandó

- Legyen L egy C2L lista fejelemére mutató pointer.
- A lista egész számokat tartalmaz, rendezzük minimum kiválasztó algoritmussal.
- Ötlet:
 - Keressük meg a minimális kulcsú elemet, és fűzzük ki a listából.
 - Első esetben a fejelem után kell befűzni, később mindig az utoljára befűzött elem mögé.
 - Így a lista elején kezd kialakulni a rendezett rész. Jegyezzük meg, hogy meddig tart ez a rendezett rész, mi az utolsó elemének a címe (legyen ez az r pointer feladata)
 - A minimum kiválasztást az r utáni elemekre kell végrehajtani, a kapott elemet r után kell befűzni, majd r-et tovább kell léptetni.
 - Ha r után már csak egy elem van, készen vagyunk.

Köszönöm a figyelmet!